

# Assignment Report

AIL861 / ELL8299 / ELL881: Advanced LLMs

**Entry No:** 2025BSZ8215

**Github URL:** [https://github.com/kushalps/llm\\_assignment1/tree/main](https://github.com/kushalps/llm_assignment1/tree/main)

**Checkpoint:** [https://huggingface.co/kushalps/tinystories/blob/main/best\\_model.pt](https://huggingface.co/kushalps/tinystories/blob/main/best_model.pt)

## 1. Objective:

Code and train a simple Transformer-based LM on TinyStories Dataset.

## 2. Dataset Description:

The dataset consists of short stories written for kids, in their vocabulary. There are about 2.1 such stories:

```
DatasetDict({  
  train: Dataset({ features: ['text'], num_rows: 2119719 })  
  validation: Dataset({ features: ['text'], num_rows: 21990 })  
})
```

## 3. Data Preprocessing:

Data consists of mix of ASCII and Unicode characters. It needs to be pre-processed to handle:

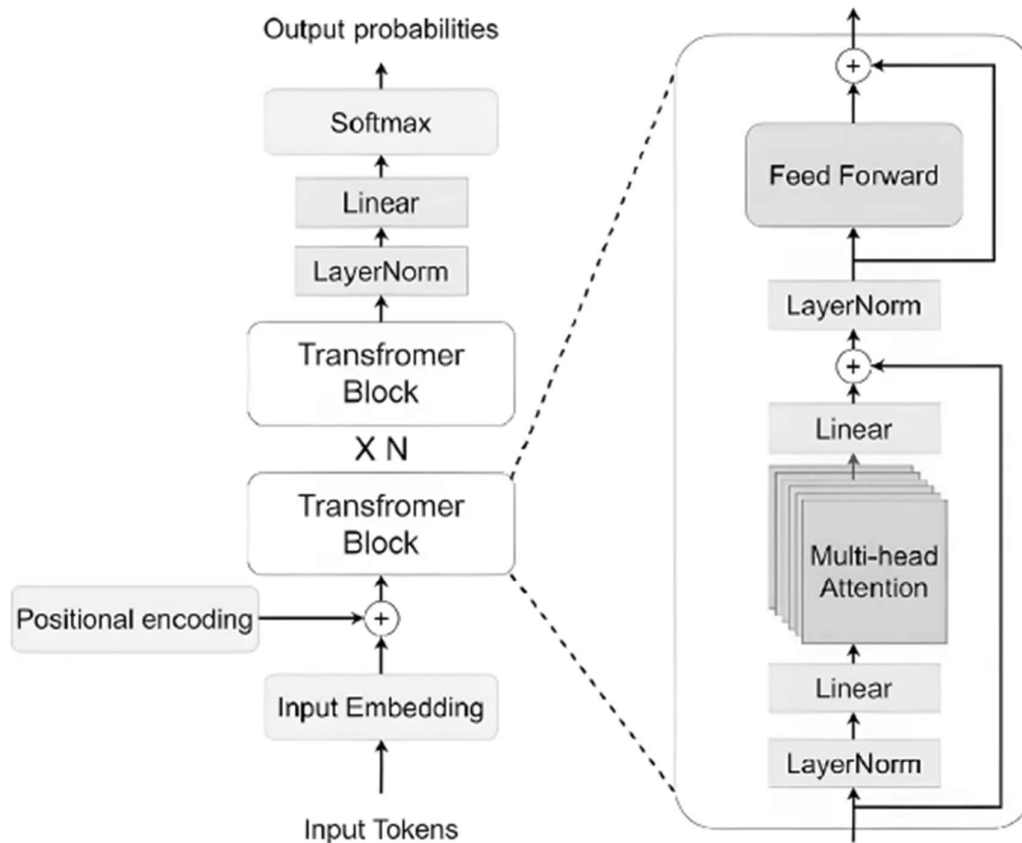
- Odd unicode (mojibake)
- Non-ASCII characters
- Long whitespaces
- Word in stuttering speech
- Word segmentation (multiple words are joined together)

The TextPreprocessor class takes care of applying these processing steps.

The words are broken along the white spaces. In vocabulary common punctuations used for generating dialogues are maintained as separate words. As a single threaded script was taking very long time to process, it was converted into a multi-threaded script. Also, on analysis I found that the spelling correction was taking long time. So, to save time, spelling correction code was removed.

## 4. Transformer Implementation

The transformer has been implemented as per the given architecture in the assignment. I started with a basic implementation. Then to speed up the code, applied constructs from Python best practices like, using static methods, pre-compilation etc.



## 5. Training and Inference

Started the training with a stride of 1. I had access to RTX 3060 GPU which is relatively old and constrained GPU. The training was taking about 25 hours for one epoch.

So, I switched to stride of 32 and 64. It was still taking very long time. So, I moved my code to my workspace to run training on a A100 GPU, where I was able to complete the training relatively fast.

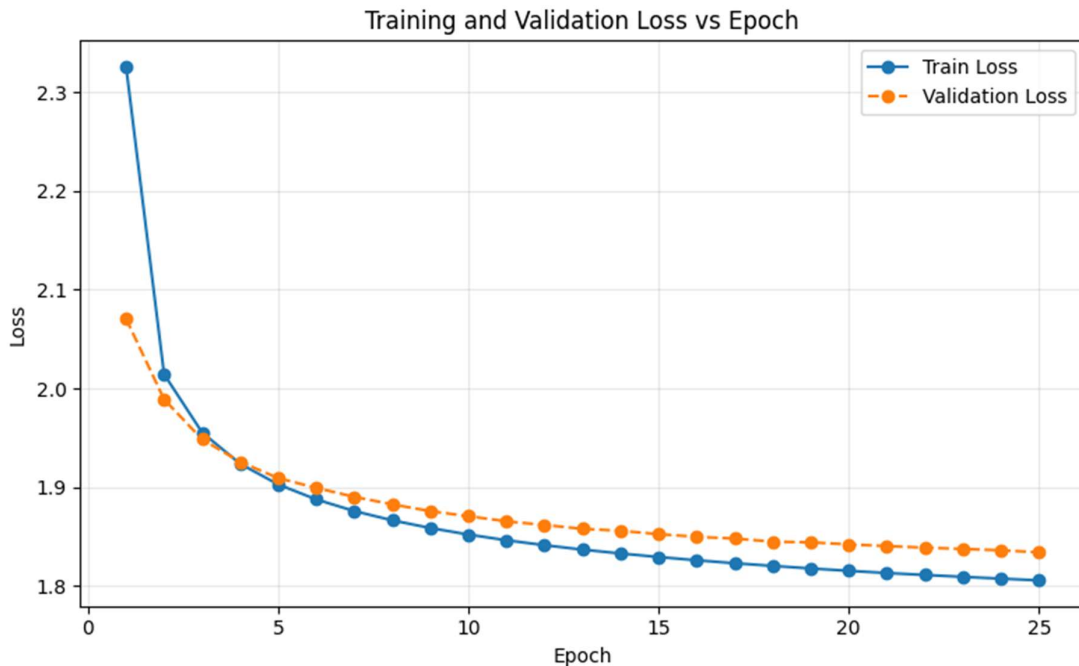
Here are the common hyper parameters:

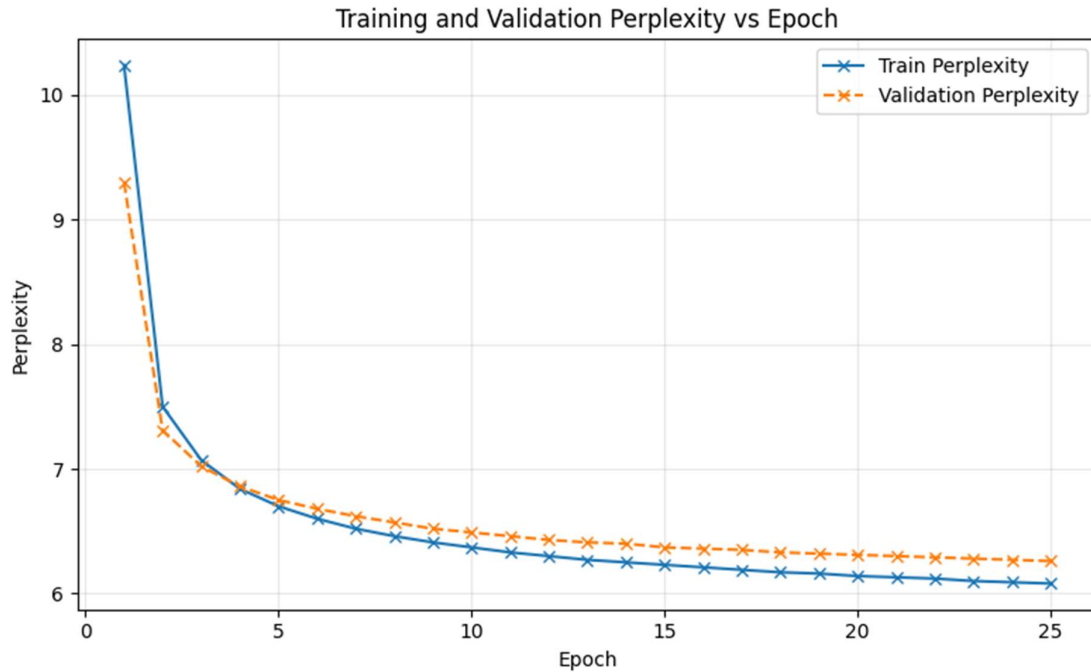
```
batch_size = 64 * 10 # 640
seq_len = 64
embed_dim = 300
num_heads = 6
num_layers = 4
lr = 3e-4
epochs = 25
vocab_size = 52130
stride = 32
```

The training log for stride 64 are given below:

[2025-11-08 10:28:34] Epoch 1/25 train\_loss=2.3265 train\_ppl=10.24 val\_loss=2.0706 val\_ppl=9.30  
 [2025-11-08 11:00:54] Epoch 2/25 train\_loss=2.0143 train\_ppl=7.50 val\_loss=1.9889 val\_ppl=7.31  
 [2025-11-08 11:33:12] Epoch 3/25 train\_loss=1.9553 train\_ppl=7.07 val\_loss=1.9483 val\_ppl=7.02  
 [2025-11-08 12:05:29] Epoch 4/25 train\_loss=1.9235 train\_ppl=6.84 val\_loss=1.9255 val\_ppl=6.86  
 [2025-11-08 12:37:47] Epoch 5/25 train\_loss=1.9026 train\_ppl=6.70 val\_loss=1.9090 val\_ppl=6.75  
 [2025-11-08 13:10:05] Epoch 6/25 train\_loss=1.8874 train\_ppl=6.60 val\_loss=1.8991 val\_ppl=6.68  
 [2025-11-08 13:42:23] Epoch 7/25 train\_loss=1.8756 train\_ppl=6.52 val\_loss=1.8900 val\_ppl=6.62  
 [2025-11-08 14:14:41] Epoch 8/25 train\_loss=1.8662 train\_ppl=6.46 val\_loss=1.8824 val\_ppl=6.57  
 [2025-11-08 14:47:00] Epoch 9/25 train\_loss=1.8584 train\_ppl=6.41 val\_loss=1.8755 val\_ppl=6.52  
 [2025-11-08 15:19:19] Epoch 10/25 train\_loss=1.8518 train\_ppl=6.37 val\_loss=1.8704 val\_ppl=6.49  
 [2025-11-08 15:51:32] Epoch 11/25 train\_loss=1.8460 train\_ppl=6.33 val\_loss=1.8652 val\_ppl=6.46  
 [2025-11-08 16:23:54] Epoch 12/25 train\_loss=1.8410 train\_ppl=6.30 val\_loss=1.8615 val\_ppl=6.43  
 [2025-11-08 16:56:12] Epoch 13/25 train\_loss=1.8365 train\_ppl=6.27 val\_loss=1.8576 val\_ppl=6.41  
 [2025-11-08 17:28:28] Epoch 14/25 train\_loss=1.8326 train\_ppl=6.25 val\_loss=1.8556 val\_ppl=6.40  
 [2025-11-08 18:00:45] Epoch 15/25 train\_loss=1.8290 train\_ppl=6.23 val\_loss=1.8522 val\_ppl=6.37  
 [2025-11-08 18:33:02] Epoch 16/25 train\_loss=1.8257 train\_ppl=6.21 val\_loss=1.8495 val\_ppl=6.36  
 [2025-11-08 19:05:18] Epoch 17/25 train\_loss=1.8227 train\_ppl=6.19 val\_loss=1.8477 val\_ppl=6.35  
 [2025-11-08 19:37:35] Epoch 18/25 train\_loss=1.8200 train\_ppl=6.17 val\_loss=1.8446 val\_ppl=6.33  
 [2025-11-08 20:09:51] Epoch 19/25 train\_loss=1.8174 train\_ppl=6.16 val\_loss=1.8439 val\_ppl=6.32  
 [2025-11-08 20:42:06] Epoch 20/25 train\_loss=1.8150 train\_ppl=6.14 val\_loss=1.8418 val\_ppl=6.31  
 [2025-11-08 21:14:22] Epoch 21/25 train\_loss=1.8128 train\_ppl=6.13 val\_loss=1.8401 val\_ppl=6.30  
 [2025-11-08 21:46:37] Epoch 22/25 train\_loss=1.8108 train\_ppl=6.12 val\_loss=1.8385 val\_ppl=6.29  
 [2025-11-08 22:18:53] Epoch 23/25 train\_loss=1.8089 train\_ppl=6.10 val\_loss=1.8372 val\_ppl=6.28  
 [2025-11-08 22:51:08] Epoch 24/25 train\_loss=1.8071 train\_ppl=6.09 val\_loss=1.8358 val\_ppl=6.27  
 [2025-11-08 23:23:23] Epoch 25/25 train\_loss=1.8053 train\_ppl=6.08 val\_loss=1.8338 val\_ppl=6.26

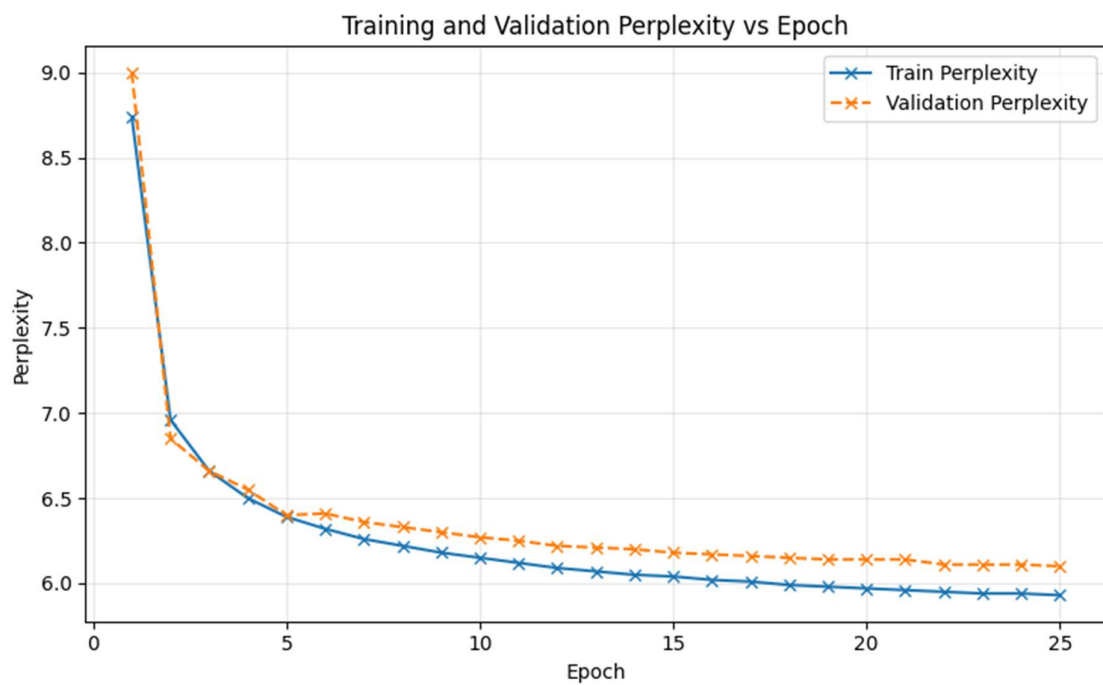
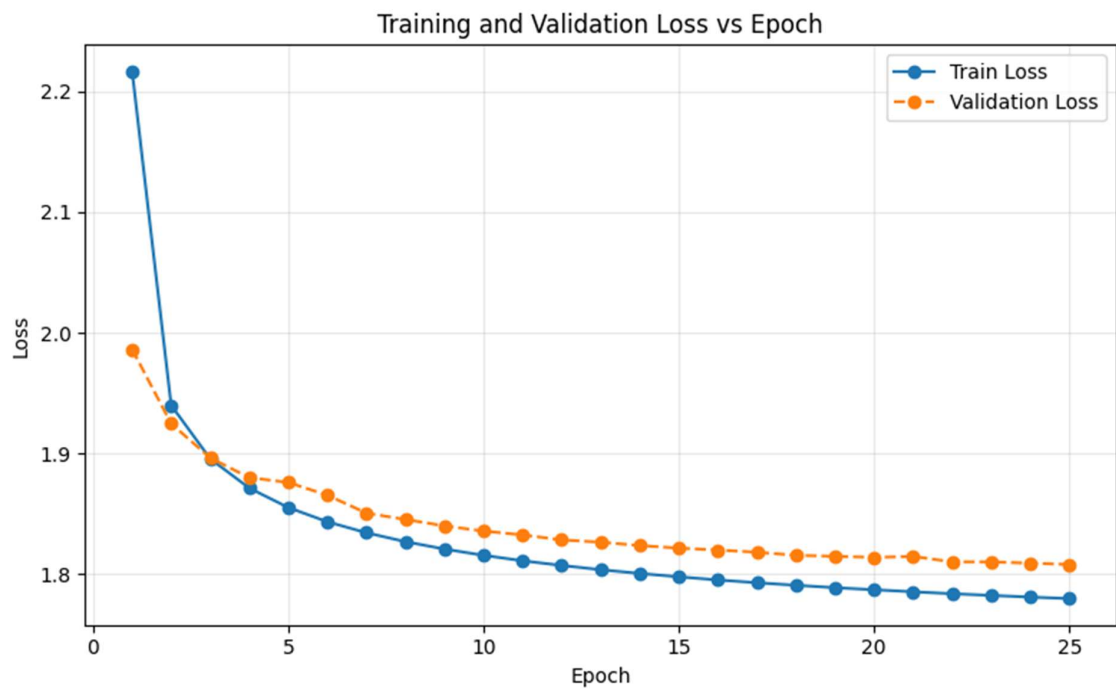
Here is the plot of training and validation errors and perplexity for stride = 64.





With stride 32, the training time was obviously higher, but the learning was better as evident from the artifacts below:

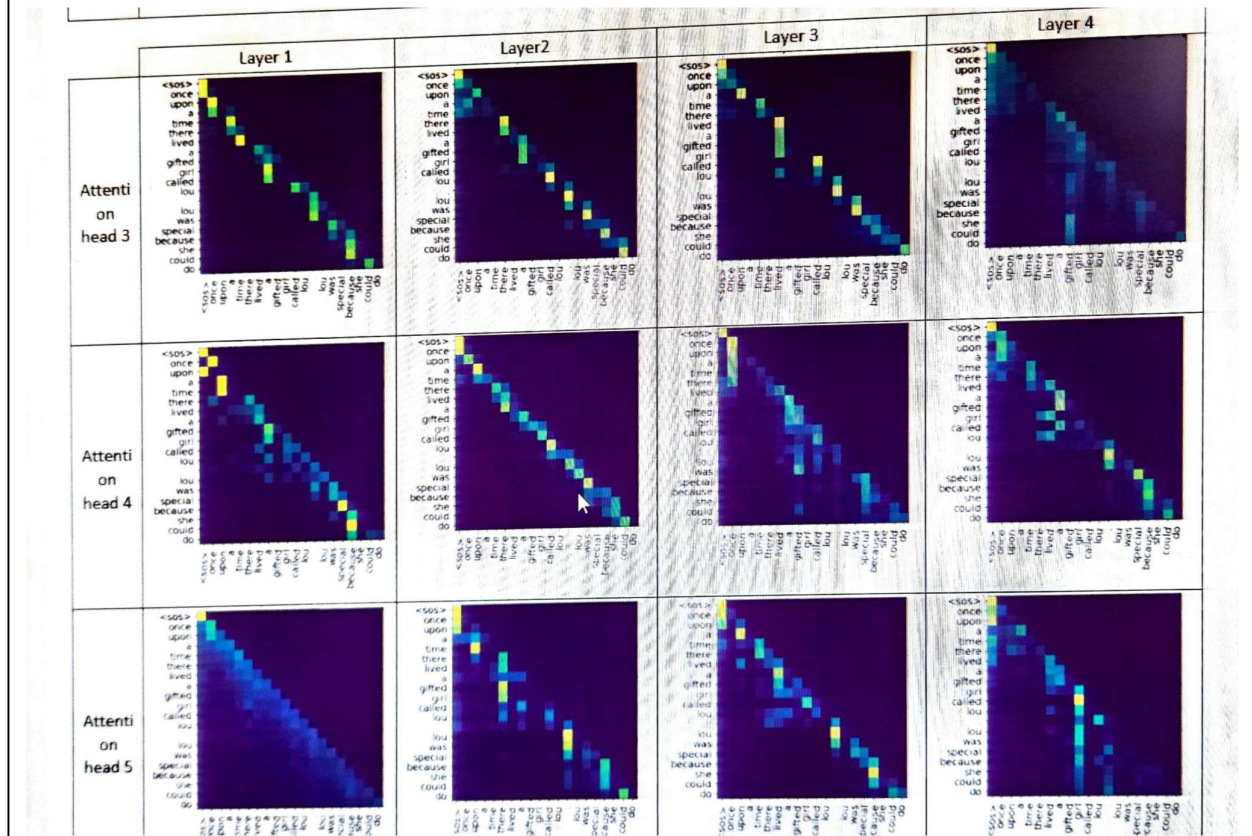
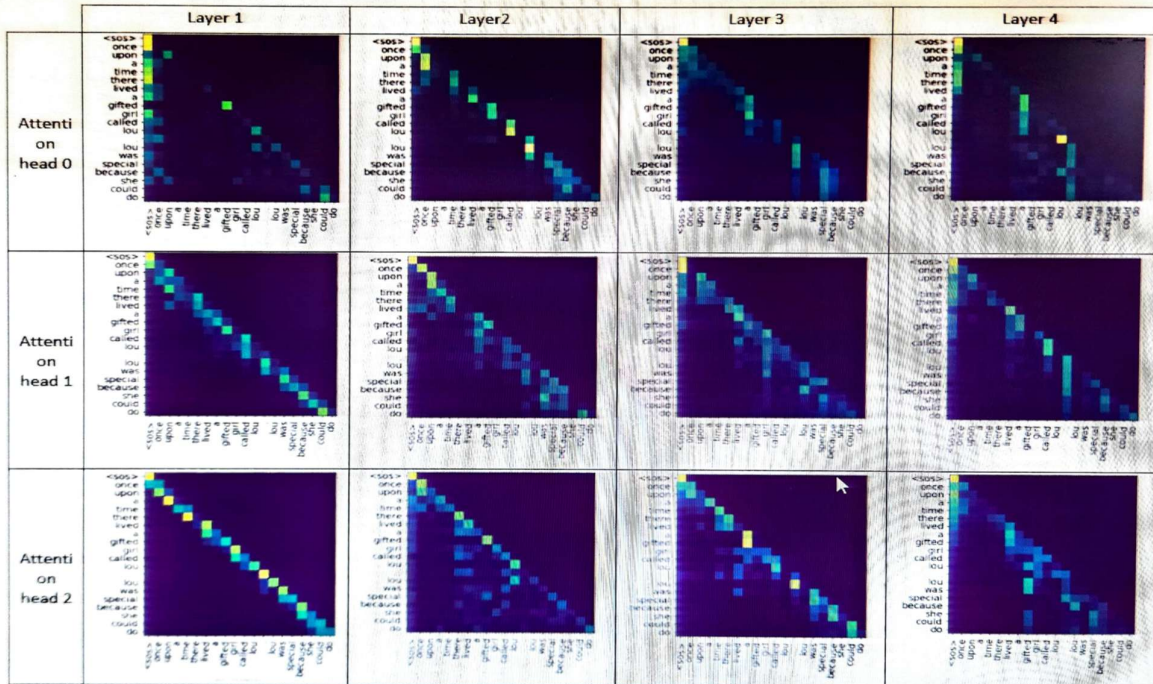
```
[2025-11-09 17:37:05] Epoch 1/25 train_loss=2.2168 train_ppl=8.74 val_loss=1.9864 val_ppl=9.00
[2025-11-09 18:41:40] Epoch 2/25 train_loss=1.9400 train_ppl=6.96 val_loss=1.9248 val_ppl=6.85
[2025-11-09 19:46:16] Epoch 3/25 train_loss=1.8957 train_ppl=6.66 val_loss=1.8964 val_ppl=6.66
[2025-11-09 20:50:49] Epoch 4/25 train_loss=1.8714 train_ppl=6.50 val_loss=1.8801 val_ppl=6.55
[2025-11-09 21:55:23] Epoch 5/25 train_loss=1.8553 train_ppl=6.39 val_loss=1.8761 val_ppl=6.40
[2025-11-09 22:59:55] Epoch 6/25 train_loss=1.8435 train_ppl=6.32 val_loss=1.8655 val_ppl=6.41
[2025-11-10 00:04:27] Epoch 7/25 train_loss=1.8344 train_ppl=6.26 val_loss=1.8507 val_ppl=6.36
[2025-11-10 01:08:58] Epoch 8/25 train_loss=1.8270 train_ppl=6.22 val_loss=1.8454 val_ppl=6.33
[2025-11-10 02:13:29] Epoch 9/25 train_loss=1.8209 train_ppl=6.18 val_loss=1.8401 val_ppl=6.30
[2025-11-10 03:17:58] Epoch 10/25 train_loss=1.8157 train_ppl=6.15 val_loss=1.8358 val_ppl=6.27
[2025-11-10 04:22:27] Epoch 11/25 train_loss=1.8112 train_ppl=6.12 val_loss=1.8326 val_ppl=6.25
[2025-11-10 05:26:56] Epoch 12/25 train_loss=1.8073 train_ppl=6.09 val_loss=1.8285 val_ppl=6.22
[2025-11-10 06:31:24] Epoch 13/25 train_loss=1.8038 train_ppl=6.07 val_loss=1.8266 val_ppl=6.21
[2025-11-10 07:35:55] Epoch 14/25 train_loss=1.8007 train_ppl=6.05 val_loss=1.8238 val_ppl=6.20
[2025-11-10 08:40:21] Epoch 15/25 train_loss=1.7979 train_ppl=6.04 val_loss=1.8217 val_ppl=6.18
[2025-11-10 09:51:22] Epoch 16/25 train_loss=1.7953 train_ppl=6.02 val_loss=1.8201 val_ppl=6.17
[2025-11-10 10:57:15] Epoch 17/25 train_loss=1.7930 train_ppl=6.01 val_loss=1.8183 val_ppl=6.16
[2025-11-10 12:01:44] Epoch 18/25 train_loss=1.7909 train_ppl=5.99 val_loss=1.8157 val_ppl=6.15
[2025-11-10 13:06:09] Epoch 19/25 train_loss=1.7889 train_ppl=5.98 val_loss=1.8149 val_ppl=6.14
[2025-11-10 14:10:35] Epoch 20/25 train_loss=1.7872 train_ppl=5.97 val_loss=1.8140 val_ppl=6.14
[2025-11-10 15:15:04] Epoch 21/25 train_loss=1.7855 train_ppl=5.96 val_loss=1.8148 val_ppl=6.14
[2025-11-10 16:19:33] Epoch 22/25 train_loss=1.7839 train_ppl=5.95 val_loss=1.8103 val_ppl=6.11
[2025-11-10 17:24:00] Epoch 23/25 train_loss=1.7825 train_ppl=5.94 val_loss=1.8103 val_ppl=6.11
[2025-11-10 18:28:27] Epoch 24/25 train_loss=1.7811 train_ppl=5.94 val_loss=1.8092 val_ppl=6.11
[2025-11-10 19:33:00] Epoch 25/25 train_loss=1.7799 train_ppl=5.93 val_loss=1.8081 val_ppl=6.10
```



**Attention Visualization:**

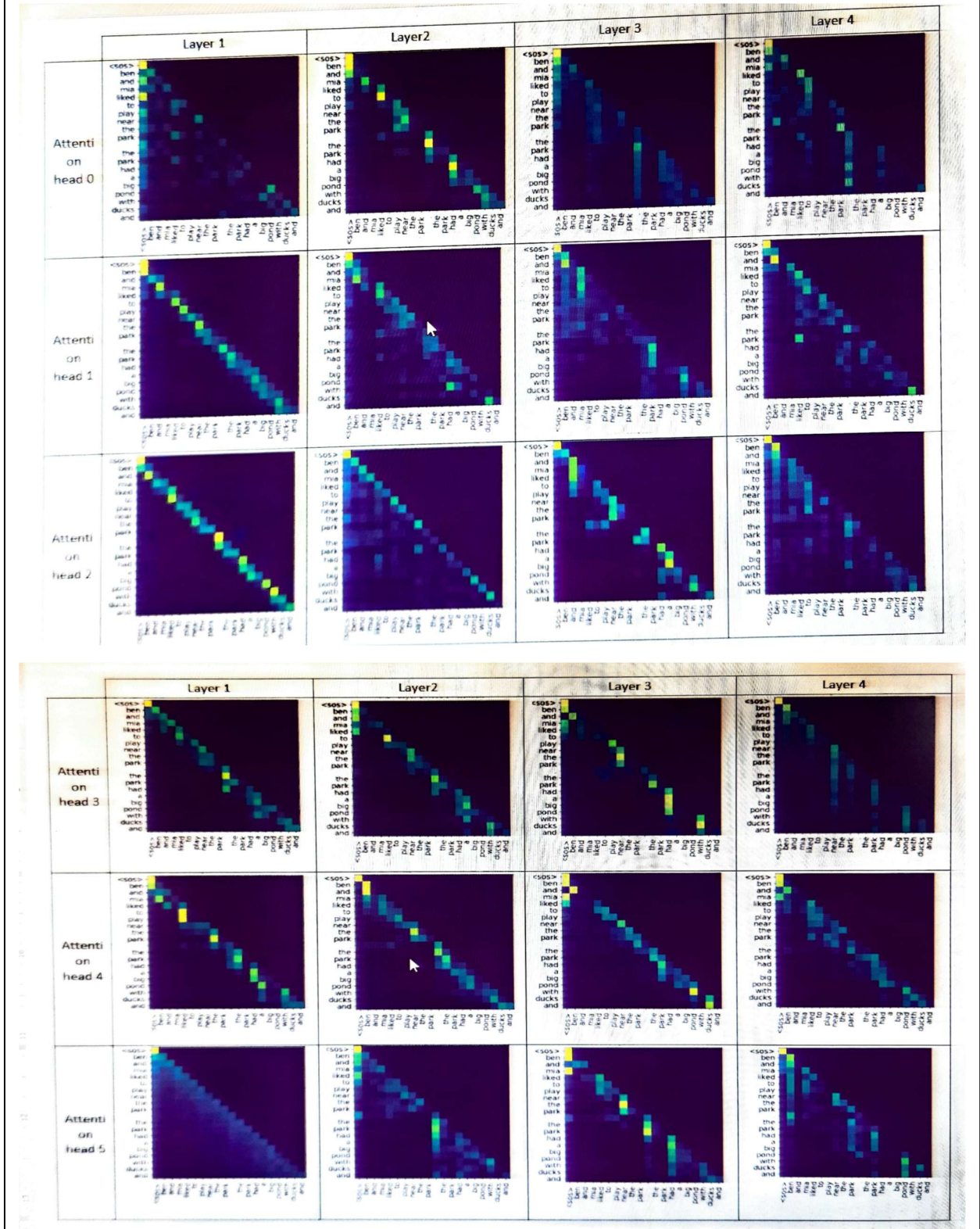
Attention Visualization on 3 random samples, for individual attention heads and layers are given below. In the attention heatmap, the x-axis represents keys and y-axis

Example 1:

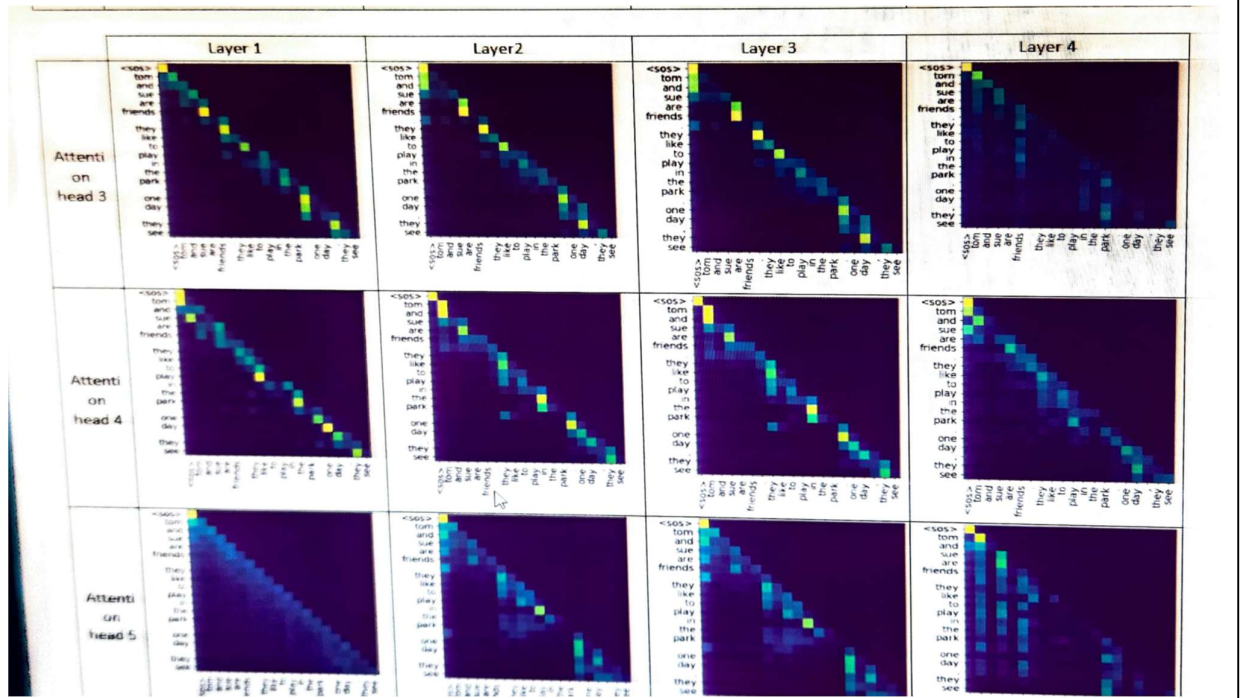
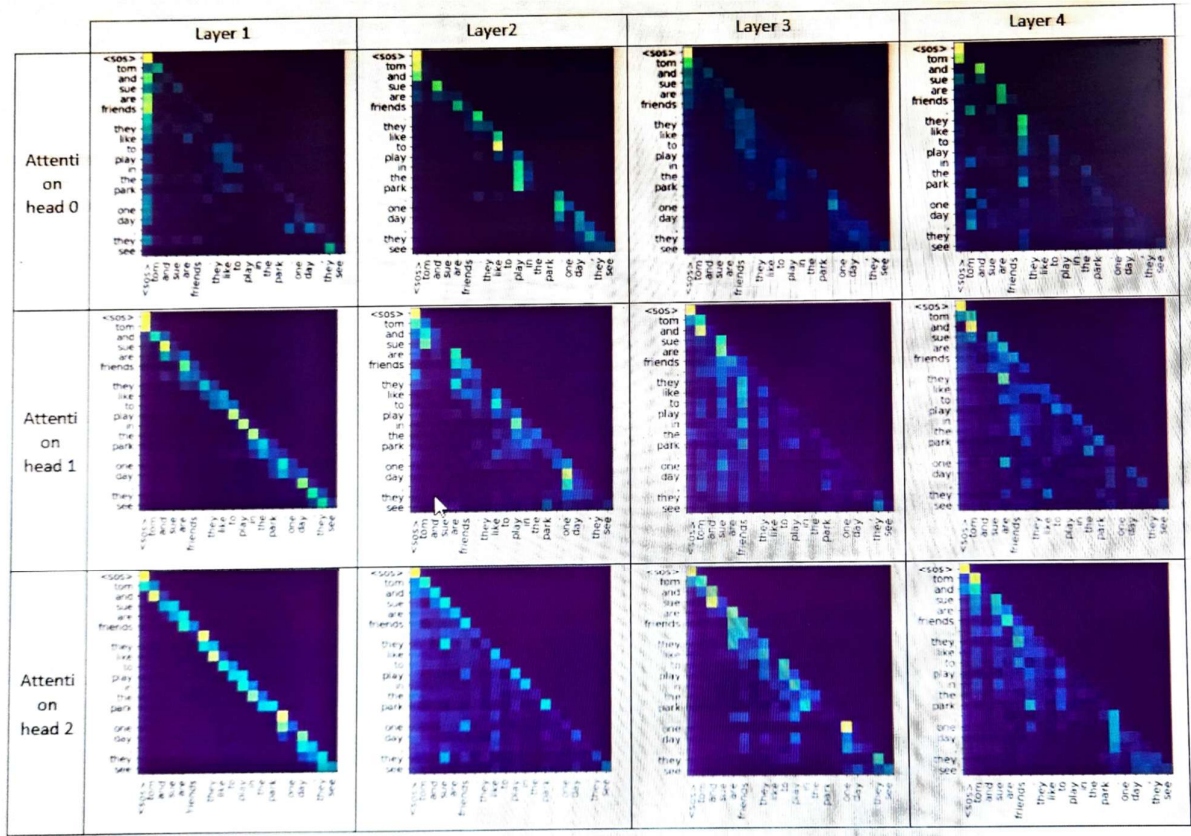


represents queries. The quality of heatmap is poor due to some technical issue. I am preparing the better version of these plots and share separately.

Example 2:



Example 3:



## 6. Greedy vs Beam Search:

Compared the generation results of greedy search with beam searched with beams of k =5 and k =10. The result is as below:

```
[greedy_search] sample 1: 50 tokens in 0.487s (102.6 tok/s), BLEU=0.1712
[greedy_search] sample 2: 50 tokens in 0.193s (259.5 tok/s), BLEU=0.0000
[greedy_search] sample 3: 50 tokens in 0.189s (265.1 tok/s), BLEU=0.0683
[greedy_search] sample 4: 50 tokens in 0.188s (266.1 tok/s), BLEU=0.0000
[greedy_search] sample 5: 50 tokens in 0.193s (259.6 tok/s), BLEU=0.1211

[beam_search_k5] sample 1: 50 tokens in 1.414s (35.4 tok/s), BLEU=0.2442
[beam_search_k5] sample 2: 50 tokens in 1.335s (37.5 tok/s), BLEU=0.0887
[beam_search_k5] sample 3: 50 tokens in 1.345s (37.2 tok/s), BLEU=0.0000
[beam_search_k5] sample 4: 50 tokens in 1.340s (37.3 tok/s), BLEU=0.0000
[beam_search_k5] sample 5: 50 tokens in 1.334s (37.5 tok/s), BLEU=0.0000

[beam_search_k10] sample 1: 50 tokens in 2.752s (18.2 tok/s), BLEU=0.3958
[beam_search_k10] sample 2: 50 tokens in 2.741s (18.2 tok/s), BLEU=0.0734
[beam_search_k10] sample 3: 50 tokens in 2.746s (18.2 tok/s), BLEU=0.0000
[beam_search_k10] sample 4: 50 tokens in 2.742s (18.2 tok/s), BLEU=0.0000
[beam_search_k10] sample 5: 50 tokens in 2.735s (18.3 tok/s), BLEU=0.0000

Summary:
decoder, avg_tokens_per_sec, avg_bleu
greedy_search, 230.56, 0.072
beam_search_k5, 36.95, 0.067
beam_search_k10, 18.23, 0.094
```

The cost of beam search is high. There is no impact on bleu score with beam of k = 5 however it improved with k =10.

## 7. Normal vs KV Caching

Below is the comparison of runtimes with normal and KV caching:

*[greedy\_search] sample 1: 50 tokens in 0.498s (100.3 tok/s), BLEU=0.1712*  
*[greedy\_search] sample 2: 50 tokens in 0.194s (258.3 tok/s), BLEU=0.0000*  
*[greedy\_search] sample 3: 50 tokens in 0.193s (259.5 tok/s), BLEU=0.0683*  
*[greedy\_search] sample 4: 50 tokens in 0.190s (262.6 tok/s), BLEU=0.0000*  
*[greedy\_search] sample 5: 50 tokens in 0.197s (253.9 tok/s), BLEU=0.1211*

*[kv\_cache] sample 1: 50 tokens in 0.192s (260.7 tok/s), BLEU=0.0000*  
*[kv\_cache] sample 2: 50 tokens in 0.179s (280.1 tok/s), BLEU=0.0000*  
*[kv\_cache] sample 3: 50 tokens in 0.179s (278.7 tok/s), BLEU=0.0000*  
*[kv\_cache] sample 4: 50 tokens in 0.181s (276.2 tok/s), BLEU=0.0000*  
*[kv\_cache] sample 5: 50 tokens in 0.177s (281.8 tok/s), BLEU=0.0000*

*Averages:*

*normal, avg tok/s: 226.9, avg BLEU: 0.0721*

*kv\_cache , avg tok/s: 275.5, avg BLEU: 0.0000*

With KV caching, the average tokens generated per second increased from 226.9 to 275.5 which translates to an increase of 21.42%.